

SEOS-2620 - Concept for splitting test_network_api

Page Content

- [Page Content](#)
- [Current state](#)
 - [Overview](#)
 - [Architecture](#)
 - [Test cases](#)
- [Proposed tests](#)
 - [Unit tests](#)
 - [System tests](#)
- [Implementation](#)
- [Proposed split into separate test systems](#)
 - [Split of the existing integration tests](#)
 - [test_network_api_tcp_client_single_socket](#)
 - [test_network_api_tcp_server](#)
 - [test_network_api_udp_server](#)
 - [test_network_api_tcp_client_multiple_socket](#)
 - [test_network_api_tcp_client_multiple_clients](#)
 - [Future proposed test systems:](#)
 - [test_network_api_unit_tests](#)
 - [test_network_api_tcp_3-9server_4client](#) (TBD, future extension)
- [Improvement suggestion - Introducing an embedded test system handler](#)
- [Decision](#)

[SEOS-2620](#) - Getting issue details...

STATUS

Current state

Overview

We have a test system with two test applications running independently from one another. This means that all resources required (for the network driver, network stack, ChanMux bandwidth) are doubled. Also due to the lack of a feedback channel between the system under test and the test environment, the synchronization between the two is implicit, creating a number of limitations.

Architecture

The test system consists of three components. The first component (test_app) implements the actual tests, the network stack component implements the actual network functionality and the driver component provides access to the actual network. This component can either be an actual network driver or our ChanMux implementation.

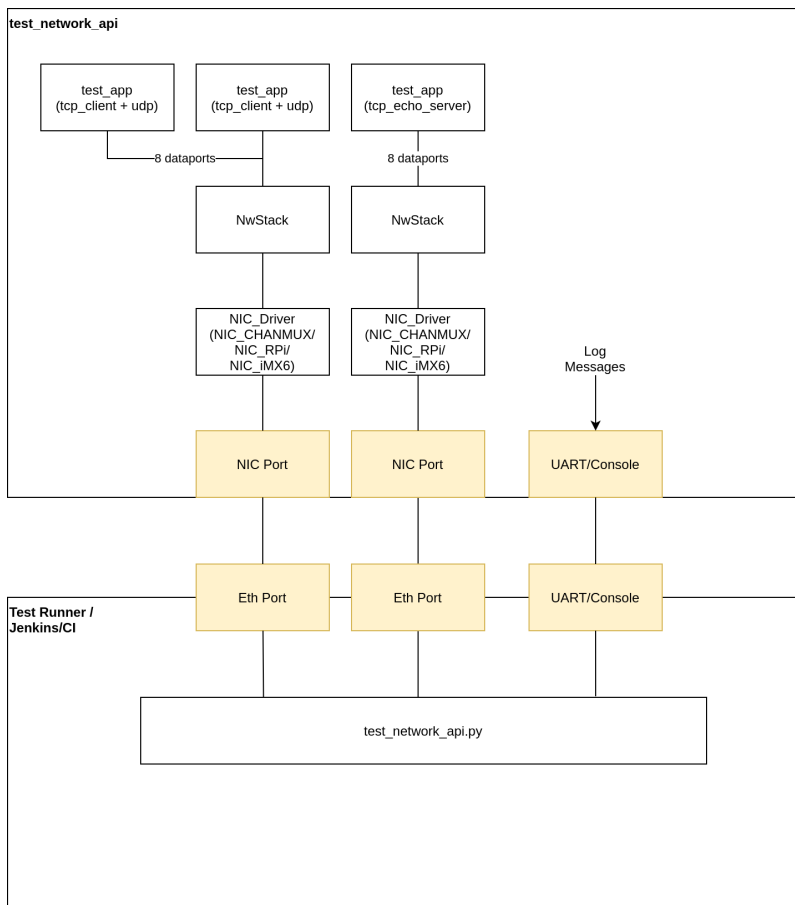
The connections between each of the components (test_app /NwStack and NwStack/NIC Driver) are implemented as RPC interfaces. These interfaces are the if_OS_Socket interface (between the test application and the network stack) and the if_OS_NIC interface (between the network stack and the network driver).

if_OS_Socket

- uses a dataport for each configured handle, the dataport is used bidirectionally
- due to the fact that multiple components can share a network stack, the serialization of the RPC calls can affect the functionality of the connected user/test components, leading to possible locks

if_OS_NIC

- uses 2 dataports, one for transmit and one for receive
- since it's connected to a single component (network stack), there are no deadlock risks here



Test cases

#	Name	Content	Environment	Expected Result	Specifics	Status	ToDo:
1	test_network_k_dataport_size_check_client	The User application calls the OS_Network functions read/recvfrom/write/sendto with invalid buffer sizes. This checks error detection on the client side of the rpc interface.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User Task calls the OS_Network function with an invalid size parameter	OS_ERROR_INVALID_PARAMETER returned by the call.	none	implemented	move to test_network_k_api_tcp_client sub test step 'dataport_size_check_client_neg'
2	test_network_k_dataport_size_check_lib	The User application calls the OS_Network functions read/recvfrom/write/sendto with invalid buffer sizes. This checks for error detection on the network stack side of the rpc interface	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User Task calls the OS_Network rpc function with an invalid size parameter	OS_ERROR_INVALID_PARAMETER returned by the call.	none	implemented	move to test_network_k_api_tcp_client sub test step 'dataport_size_check_lib_neg'

3	test_socket_create_neg	Creates sockets with invalid protocol type, domains, destination IP and ports	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User Task calls the OS_NetworkSocket_create with various wrong parameters	OS_ERROR_NETWORK_K_PROTO_NO_SUPPORT OS_ERROR_INVALID_PARAMETER OS_ERROR_NETWORK_K_HOST_UNREACHABLE OS_ERROR_NETWORK_K_CONN_REFUSED OS_ERROR_INSUFFICIENT_SPACE returned by the function call	none	implemented	move to test_network_k_api_tcp_client sub test step 'test_socket_create_neg'
4	test_socket_create_pos	Creates and closes OS_NETWORK_MAXIMUM_SOCKET_NO sockets twice	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application calls the OS_NetworkSocket_create	OS_SUCCESS returned by the function call	none	implemented	move to test_network_k_api_tcp_client sub test step 'test_socket_create_pos'
5	test_network_k_basic	Pings the test container gateway to check that docker and scapy are running as expected	Doesn't use TRENTOS	Got 5 ping replies from the gateway	none	implemented	move to all test systems
6	test_tcp_options_poison	Test for CVE-2020-24337, send a malformed packet with TCP options set to "0xFF 0x00 0x00 0x00".	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts client connections	Picotcp responds with SYNACK	none	implemented	move to test_network_k_api_tcp_server sub test step 'test_tcp_options_poison_pos'
7	test_tcp_header_length_poison	Test for CVE-2020-24341, send a malformed packet with the invalid TCP header length "0xF".	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts client connections	The malformed SYN will be dropped and opening a connection will result in a timeout	none	implemented	move to test_network_k_api_tcp_server sub test step 'test_tcp_header_length_poison_pos'
8	test_network_k_picotcp_unit_tests	The existing picotcp unit tests are integrated in our test suite	none	All tests pass	Not implemented because the results can't be easily parsed and some tests are failing.	not implemented	not implemented
9	test_network_k_picotcp_smoke_tests	The existing picotcp smoke tests are integrated in our test suite	none	All tests pass	Not implemented because the results can't be easily parsed and some tests are failing.	not implemented	not implemented

10	test_network_api_client	Two user applications open a number of connections to a HTTP server and requests a number of files.	TRENTOS application with one Network Stack Component and two User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User Task calls creates a connection to an http server and downloads data from there. HTTP Server running in the test container.	Each request gets the desired data uncorrupted.	Client app checks if the requested file was corrupted. Two instances of the test app connect to the same network stack, not all requested handles can be allocated.	implemented	move to test_network_k_api_tcp_client with one socket in a single test app move to test_network_k_api_tcp_client_multiple_socket with multiple sockets in a single test app move to test_network_k_api_tcp_client_multiple_clients with multiple sockets in multiple test apps sub test step 'test_network_k_api_client'
11	test_network_k_api_echo_server[*]	The user application opens an echo server on a given port. The python test script running in the test container connects to it and sends different number of bytes.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection User application implements Echo function and sends all received data packets back to the source of the packets	The received data by the test script is identical to the send one.	none	implemented	move to test_network_k_api_tcp_server sub test step 'test_network_k_api_server'
12	test_network_k_api_bandwidth_64_Kbit	The user application opens an echo server on a given port. The python test script running in the test container connects to it and sends a known amount of data. It then measures the time it took for it to be echoed back.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection User application implements Echo function and sends all received data packets back to the source of the packets	The time needed to send the data.	none	implemented	move to test_network_k_api_tcp_server sub test step 'test_network_k_api_bandwidth_64_Kbit'

13	test_network_tcp_connection_establishment	The user application opens an echo server on a given port. The python test script running in the test container connects to it and completes a SYN /ACK handshake.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection		none	implemented	move to test_network_api_tcp_server sub test step 'test_network_tcp_connection_establishment'
14	test_network_tcp_connection_closure	The user application opens an echo server on a given port. The python test script running in the test container connects to it and sends a SYN /ACK handshake followed by a FIN.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection	The network stack handles the FIN condition correctly.	picotcp doesn't handle this case correctly	implemented	move to test_network_api_tcp_server sub test step 'test_network_tcp_connection_closure'
15	test_network_tcp_connection_reset	The user application opens an echo server on a given port. The python test script running in the test container connects to it and sends a SYN /ACK handshake followed by a RST.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection	The network stack handles the RST condition correctly.	none	implemented	move to test_network_api_tcp_server sub test step 'test_network_tcp_connection_reset'
16	test_network_tcp_connection_invalid	The user application opens an echo server on a given port. The python test script running in the test container connects to it on an invalid port.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection	ICMP control message is received.	none	implemented	move to test_network_api_tcp_server sub test step 'test_network_tcp_connection_invalid'
17	test_network_tcp_window_size_scaling	The user application opens an echo server on a given port. The python test script running in the test container connects to it and sends data with different TCP window size.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection	not implemented	not implemented	not implemented	not implemented
18	test_network_tcp_output_of_band_signaling	The user application opens an echo server on a given port. The python test script running in the test container connects to it and sends a known amount of data. It then measures the time it took for it to be echoed back.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection	not implemented	not implemented	not implemented	not implemented
19	test_network_dhcp_get_address	The user application configures the Network Stack with the usage of a DHCP server.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application starts the process to get a DHCP lease and then configures the Network stack.	not implemented	not implemented	not implemented	not implemented

20	test_network_dhcp_renew_address	The user application renews the IP address of the Network Stack with the usage of a DHCP server.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application starts the process to renew a DHCP lease and then configures the Network stack.	not implemented	not implemented	not implemented	not implemented
21	test_network_dhcp_expire_address	The user application lets the DHCP lease expire		not implemented	not implemented	not implemented	not implemented
22	test_network_arp_request	The user application sends an ARP request	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h The user application tries to connect to a known ip	not implemented	not implemented	not implemented	not implemented
23	test_network_arp_reply_client	The python test script running in the test container sends an ARP request the IP address of the TRENTOS application running the client tests	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	ARP reply from the client app's network stack.	none	implemented	move to test_network_k_api_tcp_client move to test_network_k_api_tcp_client_multiple_socket move to test_network_k_api_tcp_client_multiple_clients sub test step 'test_network_k_arp_reply' merged with 'test_network_k_arp_reply_server'
24	test_network_arp_reply_server	The python test script running in the test container sends an ARP request the IP address of the TRENTOS application running the server tests	TRENTOS application with one Network Stack Component and User Application; User application does initialisation of Network stack (IP Addr, Gateway, Nw Mask).	ARP reply from the server app's network stack.	none	implemented	move to test_network_k_api_tcp_server sub test step 'test_network_k_arp_reply' merged with 'test_network_k_arp_reply_client'
25	test_network_udp_recv_from	The user application waits for a UDP datagram send by the test script in the test container.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application waits for a UDP datagram and checks its content	The received data is correct.	none	implemented	move to test_network_k_api_udp_server sub test step 'test_network_k_udp_recvfrom'
26	test_network_udp_send_to	The user application sends a UDP datagram send by the test script in the test container.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application sends a UDP datagram.	The send data is correct.	none	implemented	move to test_network_k_api_udp_server sub test step 'test_network_k_udp_sendto'

27	test_network_ping_request	The user application registers a callback function for ping with picotcp and tries to ping the IP address of the test container.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application pings a known IP address	Ping replies from the target	none	not implemented	not implemented
28	test_network_ping_reply_client	The python test script running in the test container pings the IP address of the TRENTOS application running the client tests	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	Ping replies from the target	none	implemented	will be refactored to ping each test instance / added to each test suite. merged with test_network_ping_reply_server
29	test_network_ping_reply_server	The python test script running in the test container pings the IP address of the TRENTOS application running the server tests	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	Ping replies from the target	none	implemented	will be refactored to ping each test instance / added to each test suite. merged with test_network_ping_reply_client

Proposed tests

Unit tests

#	Name	Content	Environment	Expected Result	Status
1	OS_NetworkStackClient_init	The user application calls the function with all combinations of valid and invalid parameters. Called by the user application to initialize the resources needed for the sockets. The other calls should fail if initialization hasn't occurred.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_PARAMETER	function doesn't implement return codes. Will be removed in future refactoring of Nw stack.
2	OS_NetworkSocket_create	The user application calls the function with all combinations of valid and invalid parameters. Called by the user application to create a new socket. Should fail if the maximum number of sockets is already in use.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_PARAMETER OS_ERROR_NETWORK_PROTOCOL_SUPPORT OS_ERROR_INSUFFICIENT_SPACE pico_err2os	test_socket_create_neg() test_socket_create_pos()
3	OS_NetworkServerSocket_create	The user application calls the function with all combinations of valid and invalid parameters. Called by the user application to create a new server socket. Should fail if the maximum number of sockets is already in use.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_PARAMETER OS_ERROR_NETWORK_PROTOCOL_SUPPORT OS_ERROR_INSUFFICIENT_SPACE pico_err2os OS_ERROR_INVALID_HANDLE	test_server_socket_create_neg() test_server_socket_create_pos()

4	OS_NetworkSocket_close	The user application calls the function with all combinations of valid and invalid parameters. Called by the application to close a socket. Multiple calls to it for the same socket should be handled correctly.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE pico_err2os	test_socket_close_neg() test_socket_close_pos()
5	OS_NetworkServerSocket_close	The user application calls the function with all combinations of valid and invalid parameters. Called by the application to close a server socket. Multiple calls to it for the same socket should be handled correctly.	TRENTOS application with one Network Stack Component and User Application; User application does initialisation of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE pico_err2os	test_server_socket_close_neg() test_server_socket_close_pos()
6	OS_NetworkServerSocket_accept	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE pico_err2os	The function is only called by Network socket create and tested by it.
7	OS_NetworkSocket_bind	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE pico_err2os	The function is only called by Network socket create and tested by it.
8	OS_NetworkSocket_listen	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE pico_err2os	The function is only called by Network socket create and tested by it.
9	OS_NetworkSocket_read	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid buffers, lengths, handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE OS_ERROR_INVALID_PARAMETER OS_ERROR_CONNECTION_CLOSED pico_err2os	test_tcp_read_pos() test_tcp_read_neg()
10	OS_NetworkSocket_write	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid buffers, lengths, handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE OS_ERROR_INVALID_PARAMETER pico_err2os	test_tcp_write_pos() test_tcp_write_neg()
11	OS_NetworkSocket_recvfrom	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid buffers, lengths, handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE OS_ERROR_INVALID_PARAMETER pico_err2os	test_udp_recvfrom_pos() test_udp_recvfrom_neg()
12	OS_NetworkSocket_sendto	The user application calls the function with all combinations of valid and invalid parameters. User application should call it with invalid buffers, lengths, handles.	TRENTOS application with one Network Stack Component and User Application; User application does initialization of Network stack (IP Addr, Gateway, Nw Mask).	OS_SUCCESS OS_ERROR_INVALID_HANDLE OS_ERROR_INVALID_PARAMETER pico_err2os	test_udp_sendto_pos() test_udp_sendto_neg()

System tests

#	Name	Content	Environment	Expected Result	Status
1	UDP echo server	The user application opens a UDP echo server on a given port. The python test script running in the test container connects to it and sends a different number of bytes, compares them with what they received.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection User application implements Echo function and sends all received data packets back to the source of the packets	Received data identical to send data.	test_udp_echo
2	UDP throughput test	The user application opens an UDP echo server on a given port. The python test script running in the test container connects to it and sends a different number of bytes, compares them with what they received and measure the time it took.	TRENTOS application with one Network Stack Component and User Application; Network stack (IP Addr, Gateway, Nw Mask) is configured automatically using system_config.h User application opens Server socket and accepts one client connection User application implements Echo function and sends all received data packets back to the source of the packets	Received data identical to send data. The time it took to execute the test.	test_udp_echo

Implementation

System under test: OS Network stack component

- a small part of the code to be tested (the code validating the parameters before the RPC call) is part of the user application

Prerequisite

- a source/sink for the frames (this can be the ChanMux driver in QEMU, native driver on the board)
- a periodic tick (ticker component)
- test application (implements tests to be executed mostly unit tests)
 - will communicate/synchronize with an external test environment (pytest) to provide input stimuli

Limitations

- currently, communication between test application and test environment happens only one way (application pytest)
 - there is implicit synchronization between the two parts, but no way to control which test is currently running.
- blocking sockets being used together with the single-threaded nature of the components can lead to deadlock while testing

Proposed split into separate test systems

For the existing integration tests and for the proposed unit test, a simple test system composed of a network driver component/network stack component /test application is sufficient.

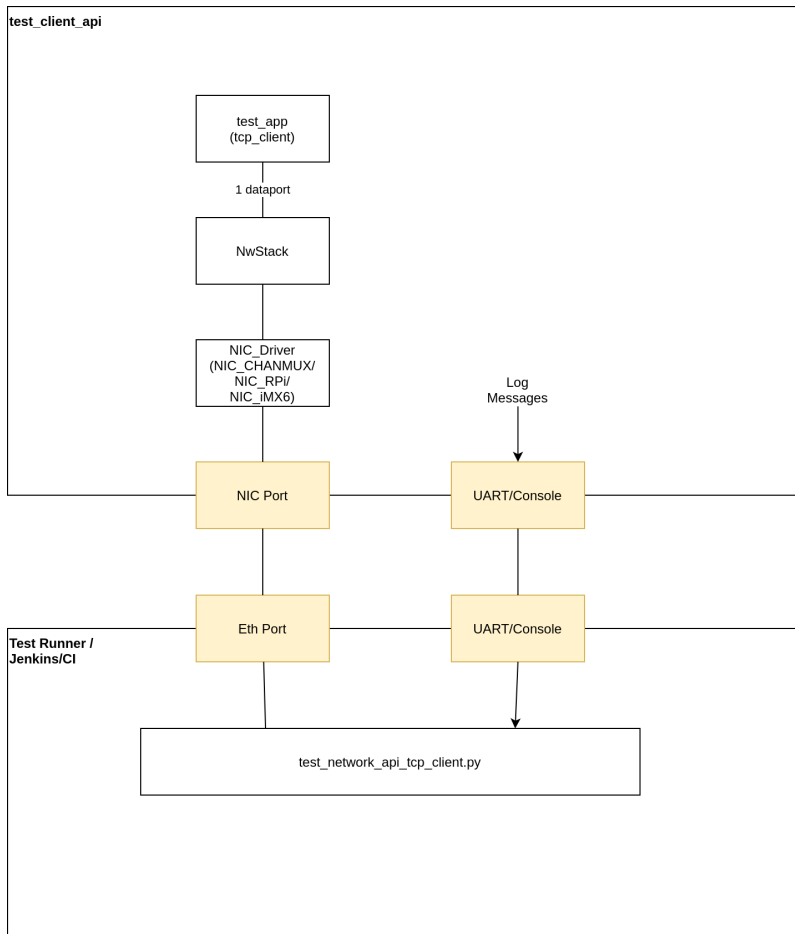
Split of the existing integration tests

test_network_api_tcp_client_single_socket

This test system shall cover the following test cases:

- TCP based tests only
- one test application
 - the test uses a single socket a single dataport is used between the network stack and the network application
 - on system start, the test application connects to some given addresses (some having a counterpart in the test environment waiting for the connection, some non-existent and expected to fail) and executes a known test sequence.
 - based on the results of the executed test sequence, the test framework is notified about the PASS/FAILED state of the test
- test case uses only TCP client socket
- Test cases:
 - test_network_ping_reply_client
 - test_network_arp_reply_client
 - test_network_api_client
 - test_network_basic

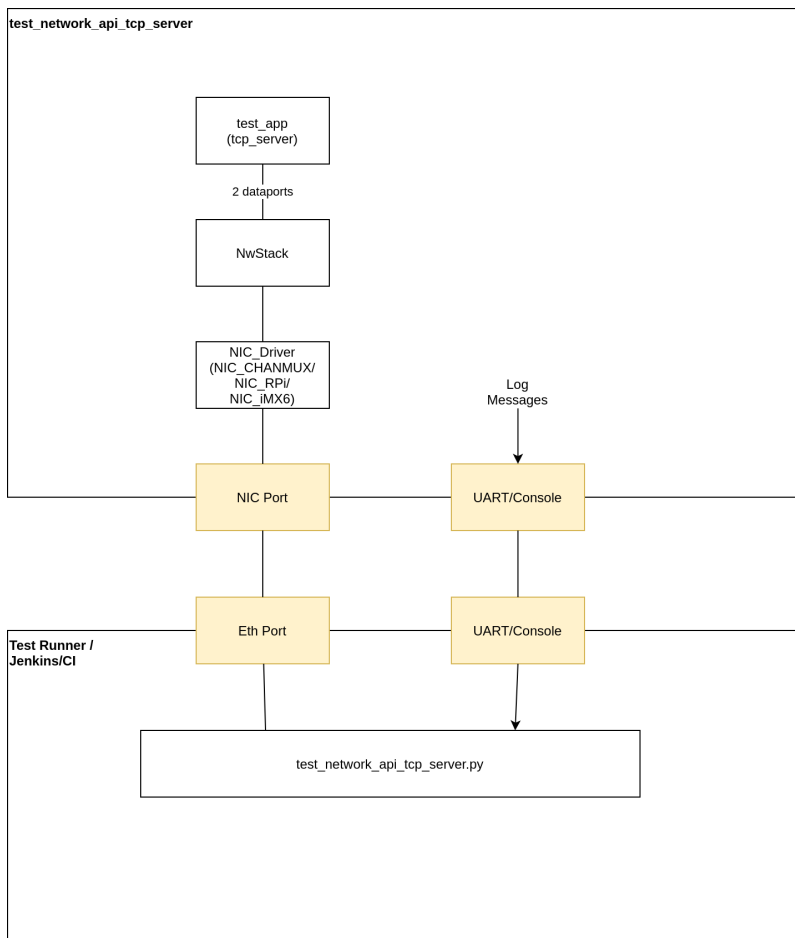
- test_socket_create_pos
- test_socket_create_neg
- test_network_dataport_size_check_lib
- test_network_dataport_size_check_client
- Infrastructure needed
 - http server
 - pytest



test_network_api_tcp_server

This test system shall cover the following test cases:

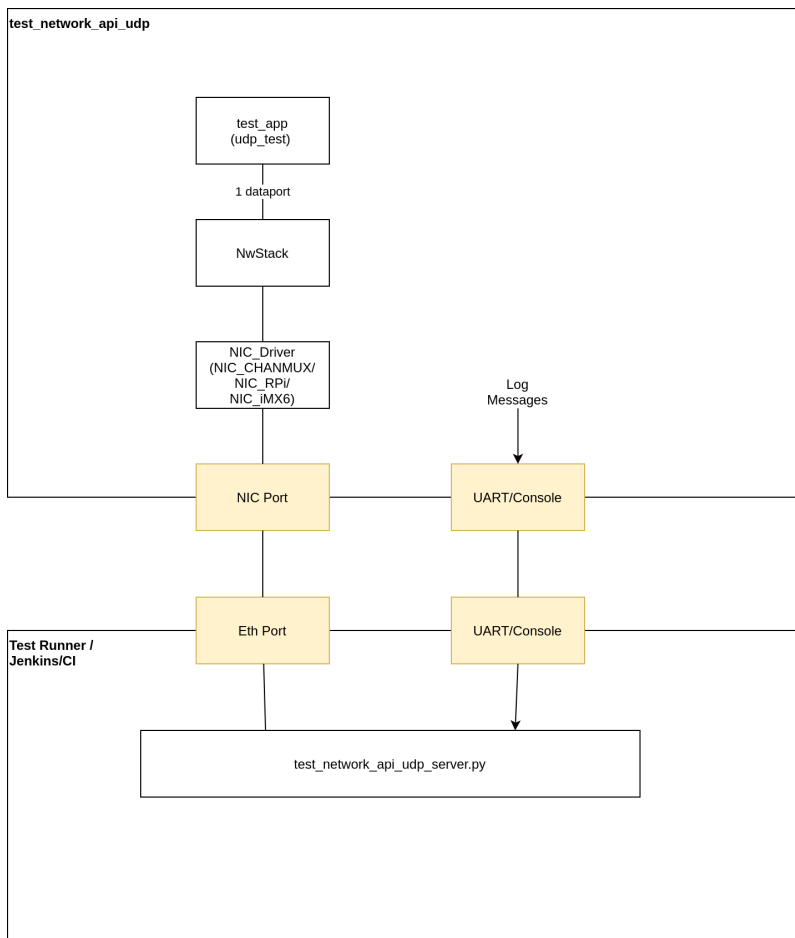
- TCP based tests only
- one test application (TCP echo server)
 - the test uses a client socket and a server socket both sockets have an attached dataport, but only the client dataport is used to actually transfer data
 - on system start, the test application starts listening on a given port and sends all received data back to its source
 - the test framework sends test-specific data, and depending on the replies received it marks the tests as PASS/FAILED.
- test case uses one TCP client socket and one TCP server socket
- Test cases:
 - test_network_ping_reply_server
 - test_network_arp_reply_server
 - test_network_basic
 - test_network_tcp_connection_establishment
 - test_network_tcp_connection_closure
 - test_network_tcp_connection_reset
 - test_network_tcp_connection_invalid
 - test_network_api_bandwidth_64_Kbit
 - test_network_api_echo_server[*]
 - test_tcp_header_length_poison
 - test_tcp_options_poison
- Infrastructure needed
 - pytest



test_network_api_udp_server

This test system shall cover the following test cases:

- UDP based tests only
- one test application (UDP echo server)
 - the test uses a single socket a single dataport is used between the network stack and the network application
 - on system start, the test application starts listening on a given port and sends all received datagrams data back to the sender.
 - the test framework sends test-specific data, and depending on the replies received it marks the tests as PASS/FAILED.
- test case uses one UDP socket
- Test cases:
 - `test_network_udp_rcvfrom`
 - `test_network_udp_sendto`
 - UDP echo test
 - UDP throughput test
- Infrastructure needed
 - `pytest`

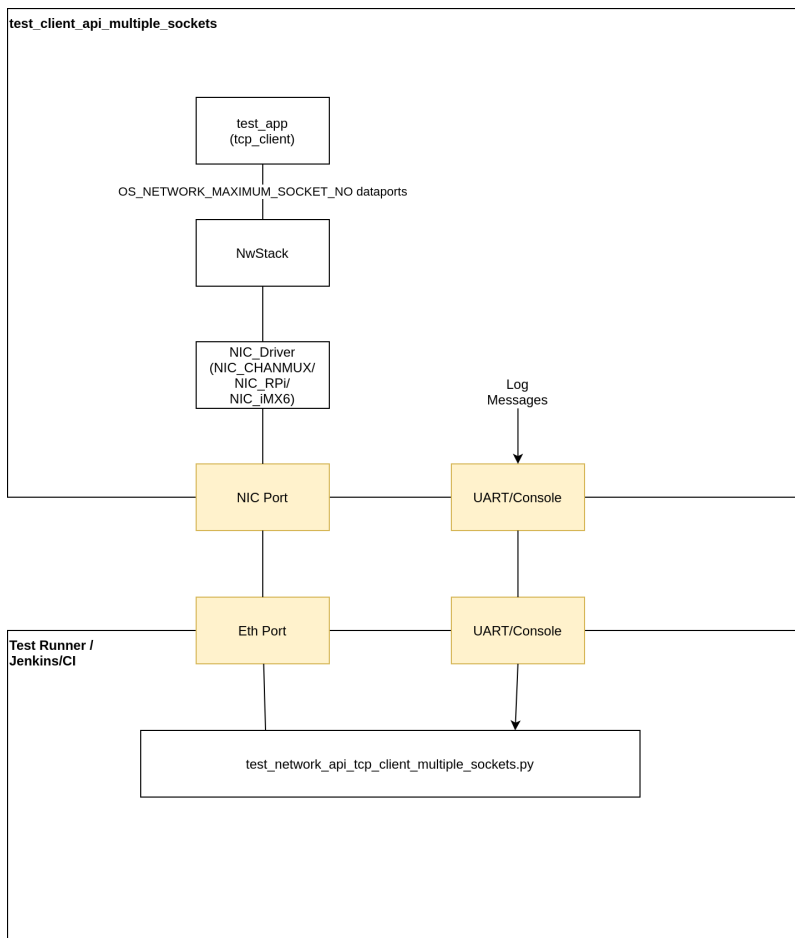


test_network_api_tcp_client_multiple_socket

This test implements a test system with multiple sockets being opened by a single test application.

This is similar to the currently implemented test_client_api integration test.

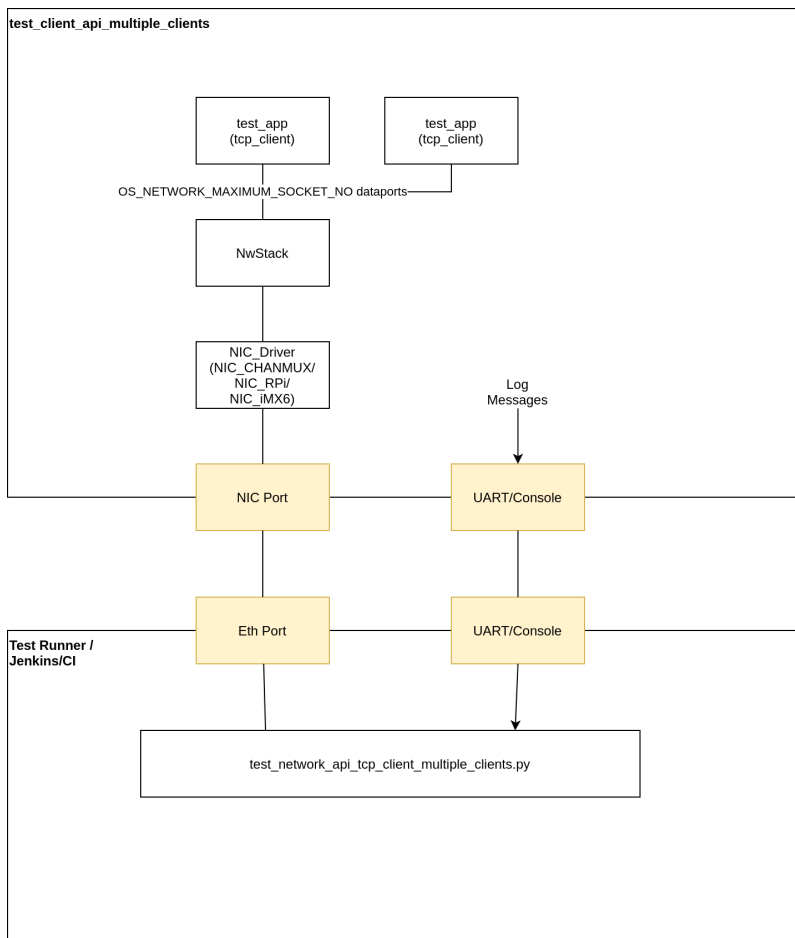
- TCP based tests only
- multiple instances of a test application
 - OS_NETWORK_MAXIMUM_SOCKET_NO is currently defined as 8
 - the test uses OS_NETWORK_MAXIMUM_SOCKET_NO socket OS_NETWORK_MAXIMUM_SOCKET_NO dataports are used between the network stack and the network application
 - on system start, the test application connects to some given addresses (some having a counterpart in the test environment waiting for the connection, some non-existent and expected to fail) and executes a known test sequence.
 - based on the results of the executed test sequence, the test framework is notified about the PASS/FAILED state of the test
- test case uses only TCP client socket
- Test cases:
 - test_network_ping_reply_client
 - test_network_arp_reply_client
 - test_network_api_client
 - test_network_basic
 - test_socket_create_pos
 - test_socket_create_neg
 - test_network_dataport_size_check_lib
 - test_network_dataport_size_check_client
- Infrastructure needed
 - http server
 - pytest



test_network_api_tcp_client_multiple_clients

This test system shall cover the following test cases:

- TCP based tests only
- multiple instances of a test application (currently 2)
 - `OS_NETWORK_MAXIMUM_SOCKET_NO` is currently defined as 8
 - the test uses `OS_NETWORK_MAXIMUM_SOCKET_NO` socket `OS_NETWORK_MAXIMUM_SOCKET_NO` dataports are used between the network stack and the network applications (all applications share the same `OS_NETWORK_MAXIMUM_SOCKET_NO` dataports).
 - on system start, the test application connects to some given addresses (some having a counterpart in the test environment waiting for the connection, some non-existent and expected to fail) and executes a known test sequence.
 - based on the results of the executed test sequence, the test framework is notified about the PASS/FAILED state of the test
- test case uses only TCP client socket
- Test cases:
 - `test_network_ping_reply_client`
 - `test_network_arp_reply_client`
 - `test_network_api_client`
 - `test_network_basic`
 - `test_socket_create_pos`
 - `test_socket_create_neg`
 - `test_network_dataport_size_check_lib`
 - `test_network_dataport_size_check_client`
- Infrastructure needed
 - http server
 - pytest



Future proposed test systems:

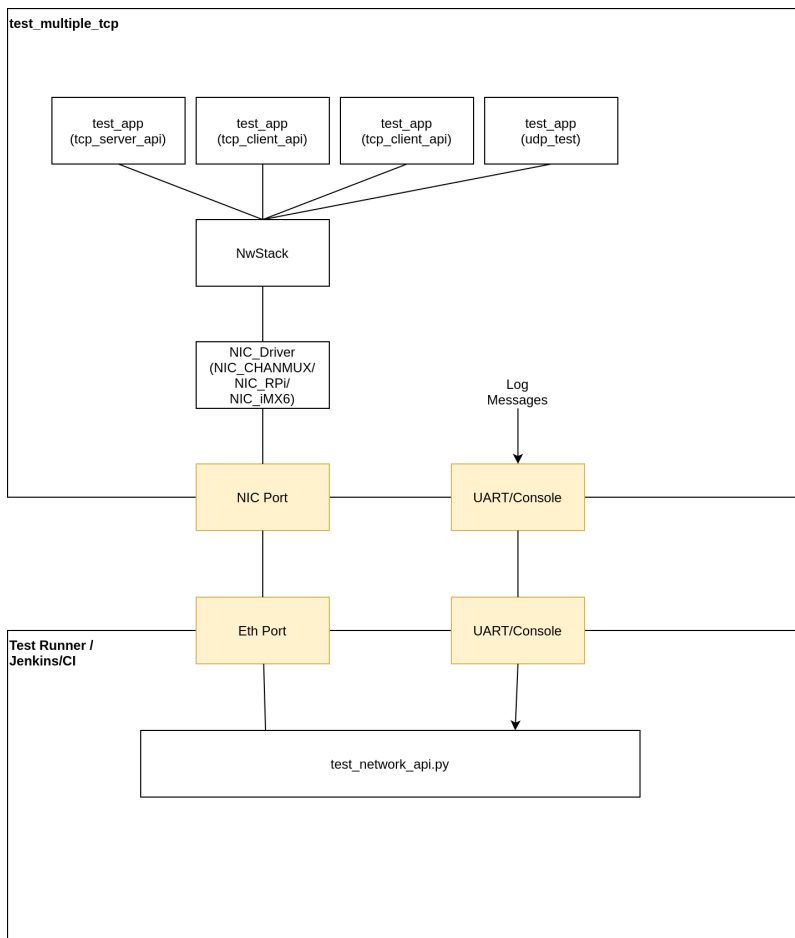
test_network_api_unit_tests

This test implements the proposed unit tests in the test application, and won't require support from the test environment.

test_network_api_tcp_3-9server_4client (TBD, future extension)

This test implements a test system having multiple test applications sharing a single network stack.

- Test cases:
 - `test_network_client_api`
 - `test_network_echo_server`
 - UDP echo server
 - all tests run in parallel in their own components
- Infrastructure needed
 - pytest
 - http server



Improvement suggestion - Introducing an embedded test system handler

This would require the implementation of a communication channel between the system under test and the test environment. This could be done either via ChanMux (for all test systems) or via the usage of UDP frames.

The UDP synchronization would make more sense for the network tests and would mean waiting for a UDP datagram with a test ID, would execute the specified test, and return the result in a different UDP datagram.

Decision

test_network_api shall be split as described in the chapters "Proposed split into separate test systems" and "Test cases".

The further proposed extension in "Future proposed test systems" and "Introducing an embedded test system handler" shall not be done within the scope of this ticket.